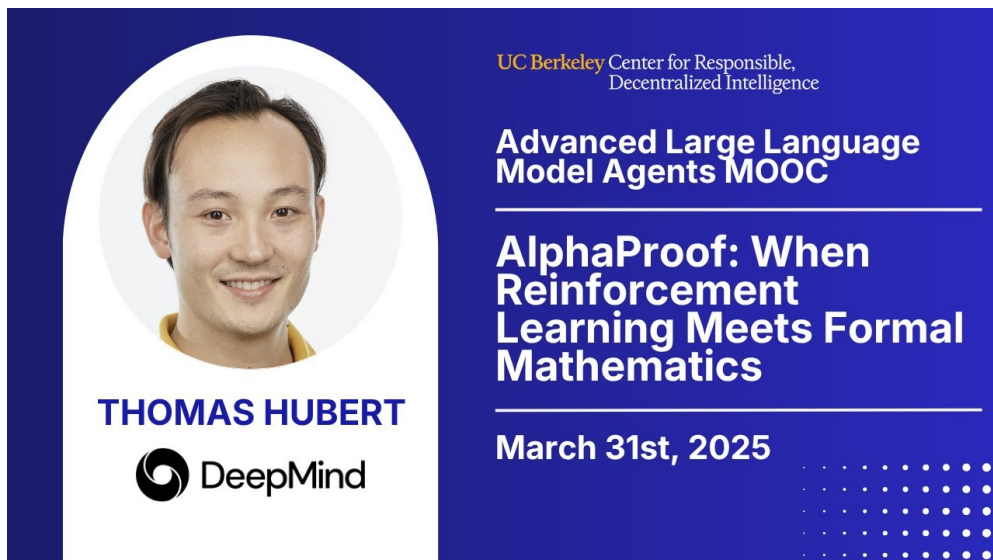


课程笔记

[LLM Agents SP25] AlphaProof: RL Meets Formal Mathematics —Thomas Hubert

基于 Thomas Hubert 授课内容整理

2025 年 3 月 10 日



视频作者/频道: Berkeley RDI

发布日期: 2025 年 3 月 7 日

视频时长: 约 60 分钟

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：从 AlphaGo 到 AlphaProof	3
1.1 本章小结	3
2 数学形式化的历史与生态	4
2.1 从希腊到计算机：形式化的世代	4
2.2 Lean、Mathlib 与规范化	4
2.3 形式化生态中的协作机制	4
2.4 本章小结	5
3 强化学习的成就与门槛	6
3.1 Zero 系列的哲学与方法	6
3.2 搜索-学习飞轮	6
3.3 探索、反馈与泛化	6
3.4 本章小结	7
4 AlphaProof: RL 遇上形式化数学	8
4.1 Lean 作为理想的 RL 环境	8
4.2 系统架构与训练 Pipeline	8
4.3 实验、IMO 2024 与迭代	8
4.4 本章小结	9
5 评测与失败模式	10
5.1 指标层级与治理	10
5.2 失败模式剖析	10
5.3 复现与稳健	10
5.4 本章小结	11
6 案例回放：Prime 定理的多策略搜索	12
6.1 问题拆解与语义化	12
6.2 策略组合与 Chunk gating	12
6.3 本章小结	12
7 工程实践与控制	13
7.1 Proof Search 调度	13
7.2 Trace 日志与验证	13
7.3 人机治理与重现	13
7.4 本章小结	13
8 知识共享与未来迭代	14
8.1 知识流与复现	14
8.2 问答亮点	14
8.3 本章小结	14

9 系统启示与未来治理	15
9.1 系统启示板	15
9.2 未来治理方向	15
9.3 本章小结	15
10 总结与延伸	16
10.1 拓展阅读	16
10.2 关键词速记	16
10.3 本章小结	17

1 引言：从 AlphaGo 到 AlphaProof

Thomas Hubert 把这场讲座定位在 Berkeley LLM Agents SP25 的“推理-执行”单元，他用 AlphaZero/AlphaTensor 把强化学习的黄金时代串联到 AlphaProof。Hubert 明确指出，数学推理所需的**严密性、证明性 feedback** 与**无限创造力**，与 RL 边界探索所需的**搜索、泛化和可靠回报**，是高度同构的。这就解释了为何 DeepMind 会把数学放在“RL 下一个战场”。

数学与 RL 的共鸣

数学是一张高维网络，节点是定理，边是逻辑链；RL 则在动作空间做探索。Hubert 强调：“只要你能把定理抽象成 action policy，理论就能交给优化器去学习。”

讲座分三段：第一段回溯数学形式化的演进并梳理 Lean/Mathlib；第二段总结 AlphaZero 的飞轮以及 AlphaProof 如何与证明系统对接；第三段聚焦工程实践、IMO 复现与治理闭环。整堂课按“理论 → 平台 → 应用”的教学逻辑展开，每段都留出时间展示经验教训与问答。

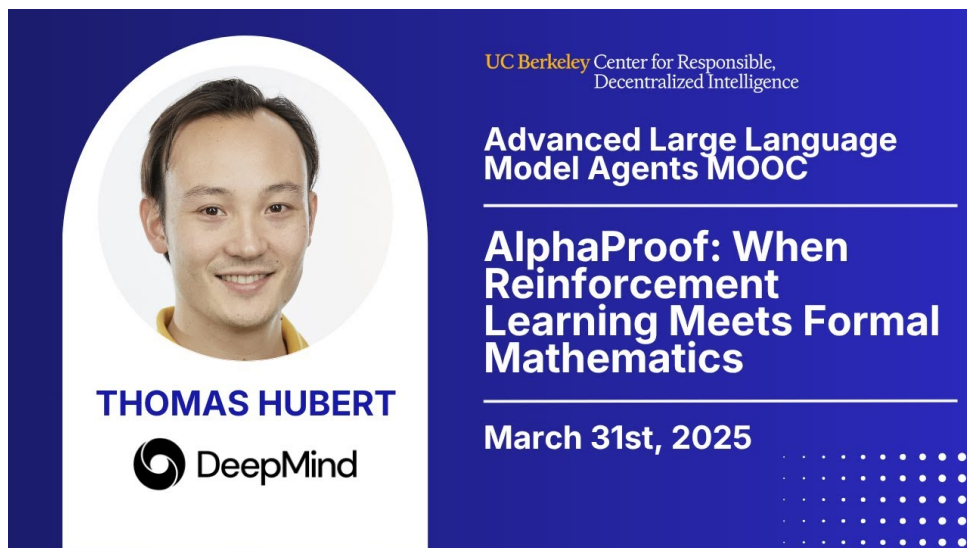


图 1: Lecture 05 封面与 AlphaProof 流程示意（封面提供主视觉/架构意象，强调 RL + 形式化的融合）。

Hubert 特别提到，这节课是 SP25 的第 5 讲，紧接在“生成式 agents 与 reasoning”之后。他把 AlphaProof 视作“数学版的 agent”，需要把 deterministic reward 和 interactive governance 结合，才能让高可信的 proof search 成为可能。

1.1 本章小结

介绍部分负责搭建语境：数学与 RL 之间的结构性相似性、AlphaProof 的使命、以及整个 SP25 讲座在“推理链 + 工程实践”之间的节奏定位，为后续内容奠定因果路径。

2 数学形式化的历史与生态

2.1 从希腊到计算机：形式化的世代

数学证明始于希腊，欧几里得开创公理化；Hilbert 将形式化写入语言。从符号化到计算时代，数学逐步借助自动化验证：Gentzen 的归纳证明、Curry-Howard 对应、Coq、Isabelle 逼近“人类可读 + 机器可验证”的交汇。

- 500 BCE：希腊几何绘制 axiom-theorem 网络；
- 1900s：Hilbert 计划与形式公理体系；
- 1970s：Curry-Howard 让类型即证明；
- 2010s：Lean 与 Mathlib 拥抱开放协作，目标“人人都能 formalize”；
- 2020s：AlphaProof 将 RL 搜索引擎接入 Lean，以应对 IMO 级别命题。

Hubert 还强调，数学形式化并不是一条单向的历史，而是“多代 parallel 更新”：Mathlib 的每个 PR 都被 replay 在 RL 环境中，RL agent 反过来又将失败 trace 反馈给 Mathlib，而 governors 在 GitHub/CI 上形成可解释的讨论，这种交叉反馈推动新公理的采纳。

2.2 Lean、Mathlib 与规范化

Lean + Mathlib 的三大原则

- 1) **可复现**：每次定理都有可编译的脚本；
- 2) **统一**：同一概念只进一个库，避免定义碎片化；
- 3) **开放**：Mathlib 通过 PR/CI 把贡献者变成治理节点。Hubert 强调：RL agent 既要读懂 Mathlib，也要对 CI 失败敏感。

这些原则直接体现在 AlphaProof 的部署中：formalizer 与 Mathlib 的 PR/CI pipeline 对齐，每次 agent 生成新的 lemma 都要先过 nightly runner；当 CI failed 时，agent 自动降级 search policy，转而将 trace 分享给 human-in-the-loop 组。

Lean 为 AlphaProof 提供的语义元素：

1. **环境 state**：当前定理、前提与 tactic 栈；
2. **动作 catalog**：不同 tactic 构成 action space，可定制 BFS、MCTS、Heuristic；
3. **反馈**：证明成功就是 +1，failure 会 trigger trace/log；
4. **Observation**：Lean 会返回 proof term、goal 树、type 约束。

2.3 形式化生态中的协作机制

Hubert 描述了 Lean 社区的协作闭环：Mathlib 维护者通过 PR/build pipeline 控制质量，AlphaProof 团队把“proof search logs + failed tactics”上传到 governance board，形成 RL trace 的输入。好几个 Lean expert 会打开 replay log 以 human-in-the-loop 方式修复 hallucination。

协作式形式化的关键节点

1) Crowdsourced replay log: 记录 agent 失败的 sequence; 2) Fairness dashboard: 验证 search 是否偏向某类策略; 3) Human review loop: 当 RL 触发 drift gate 时, expert 可 rollback knowledge base。

2.4 本章小结

数学形式化的发展、Lean/Mathlib 的规范化原则与 community-driven 协作机制共同支撑 AlphaProof 的上下文, 这一节解释了为什么把 RL 连接到 Lean 会带来“可验证的创造力”。

3 强化学习的成就与门槛

3.1 Zero 系列的哲学与方法

Hubert 把 AlphaZero 的成功归结为两个核心：**search-integrated learning** 与 **reliable reward from simulation**。在游戏中，模拟环境提供了 perfect feedback，RL 不再依赖人类 label，而是靠 search 的 wins/losses 作为 ground truth。

Zero 哲学的三个支柱

1) Self-play generating curriculum; 2) Search results distilled into policy/value; 3) Preference for small, reusable modules rather than monolithic black-box policies。

他说：“Zero 系列的辉煌不是单个模型的成功，而是一种微调过的工程流程：每一次 self-play 都会产出 curriculum，policy/value 会被重构成 search-ready 的 heuristic module。”这种哲学直接影响 AlphaProof 的 policy distillation 期望。

3.2 搜索-学习飞轮

AlphaZero/AlphaTensor 构建了一个“探索 → 学习 → 优化”的飞轮。Hubert 特别指出：如果没有 strong search，就得不到 high-quality trajectories；如果没有 fast learning，就无法 internalize 新知识；如果没有 policy/value 的紧密结合，search 会陷入 thousand-node dead-end。

飞轮缺口与潜在失败

1) Search 质量低：agent 无法发现强解，policy 更新 weak；2) 带噪声 reward：学习阶段引入 hallucinated trajectories；3) Compute 受限：search depth 不够导致 learning 无 convergence。

Hubert 把这个飞轮比作“在黑洞旁边开挖”：search 不能停留在浅层，否则 learning 永远没得 push；learning 也要及时 absorb search 发现的新 lemma，否则 search 继续重复旧路。AlphaProof 团队用 two-tier policy buffer 让 search-area 与 learning-rate 跟踪，避免出现 drift。

3.3 探索、反馈与泛化

AlphaZero 的 feedback 是 deterministic 的 win-rate；AlphaProof 必须面对“formal proof correctness”——即只要 Lean 证明不通过，feedback 就是 fail。Hubert 讲到：“在数学里，reward 要么 +1，要么 0，no gray area，这迫使 RL 发展 trace-aware regret minimization。”搜索 vs heuristics 的对比：

维度	Search-driven RL	Heuristic/Scripted
反馈质量	完美验证, replay buffer 精度高	信号噪声高, 容易 drift
泛化能力	通过 policy distillation 实现跨定理迁移	需要手工 engineering
可解释性	可以 replay trajectory, 提供 proof term	难以化简到可检查步骤

表 1: AlphaProof 的 RL 反馈与启发式方法对比

3.4 本章小结

AlphaZero 成功的三个层面 (search、feedback、learning) 成为 AlphaProof 的起点; 必须关注飞轮中的任何断裂、反馈信号的崩盘以及泛化的限制, 才能确保 RL agent 在数学世界里稳定进步。

4 AlphaProof: RL 遇上形式化数学

4.1 Lean 作为理想的 RL 环境

Lean 提供了完整的 state/action/reward/observation tuple。Hubert 演示了一个典型 episode：从 ‘goal’ 抽象出 lemma graph → agent 选择 ‘apply’/‘intro’/‘rw’ → Lean 尝试化简 goal → 成功或 fail。“我们最终把 Lean 输出当成 RL 环境的 simulator”，他如是说。

环境剖析

- 1) Action space: 分层 tactics (primitive tactic、combination tactic、search macro) ; 2) Observation: goal 树 +local context+type info; 3) Reward: 二值; 4) Reset: 每次 new theorem 重新初始化 MCTS tree。

4.2 系统架构与训练 Pipeline

AlphaProof 由多个网络组成：Formalizer 将自然语言/LaTeX 命题翻译成 Lean 断言；Proof Generator/Actor-critic 负责 search；Verifier 负责二次检查；Runner 记录 trace。

组件	责任	杂记
Formalizer	结构化命题、生成 Lean declarations	基于 seq-to-seq + retrieval
Proof Generator	RL + search 生成 tactic sequence	蒙特卡洛/beam 结合
Verifier	Lean 验证 proof term	失败即触发 rollback
Runner	记录 proof trace + metrics	用于 governance gate

表 2: AlphaProof 架构组件

Pipeline 的治理点

每个 component 都写入 governance log：Formalizer 的 semantic gap 比对、Proof Generator 的 entropy/confidence、Verifier 的 failure rate、Runner 的 chunk gating hit。Hubert 说：“只有把 trace 变成结构化 artifact，才能让 RL 不是黑箱。”

训练时，AlphaProof 在多个 dataset 上交替训练：Mathlib core definitions、IMO contest statements、AlphaGeometry proofs。Hubert 强调，每个 dataset 都对应独立 replay buffer（“Lemma buffer”），并在 training schedule 上设置 warm start → search-intensive → human eval 3 个阶段，使得 policy 愈发稳健。

4.3 实验、IMO 2024 与迭代

AlphaProof 联合 AlphaGeometry 取得 IMO 2024 银牌，主要突破在于：1) 构造了 large-scale formalization dataset；2) 借助 replay buffer 在 proof search 中记录 failure trace；3) 把 governance board 的 human review 结果当作 reward shaping。Hubert 特别展示了一个说明：当 agent 在 ‘prime’

定理上 fail 时，会把 trace 上传到 ‘AlphaProof Repos’，由 math expert 生成 corrected lemma，再 feed 回 policy。

IMO 级别的部署提醒

1) 要把 search depth、tactic mix、chunk gating 等参数切片化测试；2) 在 proof 驻留时间超过 500ms 时要强制 log；3) 不能只看 final success rate，还要 tally drift gate triggers。

4.4 本章小结

AlphaProof 通过 Lean 提供的 deterministic reward、structured observation、rich action space，把 math proof search 变成可控的 RL pipeline，治理 log 与 human review 让失败也有用。

5 评测与失败模式

5.1 指标层级与治理

指标	描述	采集频率	触发条件
Proof success rate	Lean 证明命题是否成功	real-time per episode	success < 2% → 放大 search
Entropy drift	Policy entropy drop	epoch	drop > 30% → drift gate log
Chunk hit rate	Search track cache 命中率	per batch	< 80% → adjust overlap
Human edit count	需人工干预的 trace 数量	weekly	> 5/1000 → governance review

表 3: AlphaProof 的多层评测指标与响应策略

Hubert 提到 slides 17-19 展示了整个 metric dashboard: top 部分是 proof success/distribution, 左侧是 entropy drift heatmap、右侧是 user feedback count。dashboard 直接与 governance board 的 alert pipeline 相连, 一旦某条 track cross threshold, 就自动把 replay log push 给 reviewer。

评测与治理的闭环

1) 多指标并行采集; 2) 当 proof success rate 低于 baseline 时, 增加 search depth 并 log replay; 3) Entropy drift 与 manual edits 共同决定是否需要 governance review。

5.2 失败模式剖析

AlphaProof 记录的失败多集中在以下几个 pattern: A) semantic gap: formalizer translation 错误; B) search drift: policy 进入 low-entropy zone; C) Lean constraints: type mismatch 造成 goal tree 崩塌; D) compute budget exceed。Hubert 用 “failure funnel” 形象表示: 大部分 failure 在 initial seeding 阶段被 filter, 只有少数 drift via chunk gating 到 governance board。

常见失败提醒

Semantic gap 需要回到 formalizer; drift gate 触发后先分析 entropy heatmap; 类型错误要检查 Lean goal stack; budget exceed 时直接降到 low-latency track。

5.3 复现与稳健

每个失败 trace 都写入 ‘failure-repro.json’, 包括 time stamp、search track、entropy、human comment。团队会把这些 trace 上传到 experiment repo 生成 ‘failure curriculum’, 让 RL agent 重新 warm start。这个过程由 governance board 监管, 确保每条 failure log 都能变成 next training seed。

Failure curriculum 的要素

1) Trace + entropy + comment 组成 entry; 2) Entry 进入 Lemma buffer; 3) Training schedule 将其作为 positive/negative sample; 4) Result logged back to dashboard。

5.4 本章小结

评测与失败模式层打通感知（指标）、诊断（failure taxonomy）与复现（failure curriculum），让 AlphaProof 在迭代中把失败变成可控的训练数据。

6 案例回放：Prime 定理的多策略搜索

6.1 问题拆解与语义化

Hubert 把一个 prime 定理拆解成三个子 goal: 1) 从定义出发构造候选数; 2) 用 lemma 联结 prime 与 modulus; 3) 用 tactic 列出推理 steps。每个子 goal 都在 Lean 中形成独立的 goal tree, AlphaProof 会先在 Mathlib 中查找相似 lemma, 再调用 policy 生成 tactic。

Prime 定理的语义纹理

1) 定义层: prime number 需要从 basic arithmetic 过渡到 advanced number theory; 2) 策略层: 不同 lemma 如 `mod_pow`、`order_dvd` 分别用不同 tactic; 3) 治理层: 每个 unsuccessful tactic 都写入 trace, 供 human reviewer 复审。

6.2 策略组合与 Chunk gating

在实际执行中, AlphaProof 同时维持三条 search track: heuristic-run (低成本), beam-run (高保真), MCTS-run (深度 polish)。chunk gating 表里记录了每条 track 的 token budget、overlap、entropy drift; 当 heuristic-run 勾勒出可能解时, beam-run 会接手; 若 drift 超过阈值则降级到 MCTS-run, 确保 eventual proof 的正确性。

Track	Budget	说明
Heuristic-run	128 tokens	快速探索, 生成 proof sketch; 当 heatmap 低于 0.4 后让 beam 接入
Beam-run	256 tokens	保守扩展 top-k candidate, 加入 lemmas
MCTS-run	512 tokens	深度 polish, triggered by drift gate

表 4: Prime 定理 search track 与 gating

实时调度的注意事项

Chunk gating 不能只是 threshold table, 还要根据 proof term 长度调整 budget; 如果 drift gate 频繁触发, 说明 Beam track parameter 过 aggressive。

6.3 本章小结

Prime 定理案例展现了 multi-track search 与 chunk gating gate 的配合, 让 RL policy 在数学定理上既能快速探索又不失准确性; Trace log 让 drift gate 遇到的失败成为治理的素材。

7 工程实践与控制

7.1 Proof Search 调度

Hubert 介绍了一个调度器：Edges 具有不同 cost (Heuristic, Beam, MCTS)，每次在 chunk gating 表中选择 search policy；Chunk gating 记录了 token count、Overlap 以及 proof term size。调度器根据 latency budget 选用“浅 search + high replay”或“深 MCTS + low replay”。

Search 类型	特征	适用场景
Heuristic tactics	低 latency	前期 exploration
Beam search	保守选择 top-k	lemma discovery
MCTS + RL policy	状态 value + policy	最终 polish, 适合定理级

表 5: Proof search 调度策略

调度器的 KPI

Latency budget、proof-term length、entropy drift 共同驱动 chunk gating gate；超过阈值就降级到 conservative search。

7.2 Trace 日志与验证

每次 failed proof 都会录入 ‘trace.json’：包含 tactic sequence、Lean 提示、search depth。Hubert 强调 trace 里必须有 ‘human comment’ 字段，使 governance team 能快速 triage。他还指出：Lean 验证失败后，RL 需要 replay alternate candidate 以避免 stuck。

Trace 日志的警示

如果没有 replay (trail) 就不可能衡量 strategy drift；没有 trace 的 replay buffer 会让 policy 盲目训练 wrong proofs。

7.3 人机治理与重现

治理团队会定期把 high-drift chunk 送到 human-in-the-loop board：reviewer 检查 trace/log、评估 fairness、在 newsletter 中更新 knowledge summary。这种做法让 AlphaProof 将每个失败当作复现实验的 seed。

治理要点

1) High drift chunk 触发 alert；2) Reviewer 生成 corrected tactic；3) Policy replay + reward correction；4) Newsletter 里记录 QA 亮点。

7.4 本章小结

工程部分强调 Proof search 调度、trace 记录与治理闭环：通过 chunk gating、trace + review、newsletter 三角形，把 RL agent 从未解释的 black box 变成可复现的研发生命线。

8 知识共享与未来迭代

8.1 知识流与复现

AlphaProof 团队把 experiment log、failed proof trace、human comment、newsletter 实验记入单一 repo；每个 release 都附带 ‘diff’，用来 retrain policy。Hubert 指出：“我们需要把 QA 亮点、governance decision、正负例 proof 共享给新成员，这样 training loop 才能迭代得更快。”

知识共享机制

1) Experiment repo, 包含 chunk gating journal; 2) Newsletter, 每周记录 QA + governance highlight; 3) Shared slides + transcript, 也是新成员 ramp-up 的料。

8.2 问答亮点

QA 环节被用来说明是否能将 AlphaProof 扩展到 non-Euclidean geometry 以及 hypergraph theorem。Hubert 回答时强调要把 RL policy 设计为 modular, 每次只调 search depth 或 tactic mix, 就可以逐步扩展到更抽象证明。

问答中的谨慎点

不要盲目用 AlphaProof 解决所有问题；在 low-resource 语言或 novel math branch 里, policy 需要重新 warm start, 并配合 chunk gating 与 fairness board。

8.3 本章小结

知识共享段落展示了 log/newsletter/transcript 如何形成复现网络，以及通过 QA 环节暴露的扩展挑战，进一步说明 iteration 的边界与治理需求。

9 系统启示与未来治理

9.1 系统启示板

Hubert 把 AlphaProof 的设计称为“三层可解释架构”：1) Formalizer/Proof Generator/Verifier 组成 deterministic pipeline；2) Trace/log + dashboard 构成 observability layer；3) Governance review + newsletter 则是 human-in-the-loop layer。AlphaProof 的体验告诉我们：复杂 agent 必须把 pipeline、metric、治理明确分层，才能在 math 这样没灰度的领域安全部署。

层级	关注点	做法
Pipeline	correctness + modularity	Formalizer/Proof Generator/Verifier + Runner 构成 steeped pipeline
Observability	metrics	Dashboard + chunk gating + trace log
Governance	trust	human review + newsletter + failure curriculum

表 6: AlphaProof 的系统启示三层

系统启示精髓

把 system design 视为 pipeline + observability + governance 三层架构，可以确保最高敏感的数学推理任务在未知场景也维持可解释与可控。

9.2 未来治理方向

Hubert 提出“differentiated governance”思路：对不同 search track 设立不同 drift threshold，并把 governance decision 写入 newsletter/backlog。比如 beam search 漏洞由 governance reviewer 直接 patch 相关 policy，而 heuristic track 则由 RL agent 自主重试。他强调：治理不是 judge，而是 feedback policy。

治理不要简单封杀

当 governance alert 触发，不是直接 disable agent，而是把 human comment 写回 failure curriculum、提高 reward shaping，再恢复 search；否则会让 system 变成 fragile checkbox。

9.3 本章小结

系统启示总结出三层架构，未来治理则通过 differential threshold + feedback loop 让数学 agent 继续拓展边界；这些思考也能反哺其他需要高度可解释的 RL 系统。

10 总结与延伸

模块	关键收获	下一步行动
数学形式化	Lean/Mathlib 提供 deterministic environment 以及可协作治理	把 Mathlib PR/gen replay log 纳入 experiment pipeline
强化学习	AlphaZero 的 search-feedback-learning 飞轮是 AlphaProof 的设计基座	继续强化 search-policy distillation 与 entropy gate
AlphaProof 架构	Formalizer + search + verifier + runner 形成可控 pipeline	增强 chunk gating 触发机制, 丰富 trace artifact
评测与失败	多层指标与 failure curriculum 让 failure 变成训练数据	扩充 dashboard 指标集合并自动触发 failure curriculum
工程实践	Proof search scheduling + trace logging + governance closing loop	设定 drift threshold, 并把每次 alert 视作 retraining seed
知识共享	log/newsletter/transcript 构成复现 network	让 newsletter 成为 QA + governance highlight 的 archival record
系统启示	Pipeline/observability/governance 三层架构构成信任基础	把 differential governance policy 写入 newsletter & automation

表 7: Lecture 05 的技术模块与行动计划

10.1 拓展阅读

- Hubert et al., “AlphaProof: RL Meets Formal Mathematics,” 2024。
- Lean 官方文档与 Mathlib README (GitHub)。
- AlphaZero / AlphaTensor 相关论文。
- IMO 2024 silver proofs 公开 archive。
- Terence Tao 关于形式化协作的博客。
- “Governance for autonomous agents” 的 workshop notes。

10.2 关键词速记

- Chunk gating: proof search 中的 token/overlap/latency 策略选择器。
- Drift gate: entropy 下降触发 fallback + review。
- Trace log: 每次 failed proof 的 tactic sequence + human comment。
- Knowledge pipeline: log + newsletter + transcript 三角形。

关键词速记的作用

用统一术语帮助新成员快速上手 AlphaProof 的 operational playbook，并配合 governance board 形成反馈闭环。

10.3 本章小结

AlphaProof 是一个把数学形式化、强化学习与可治理工程实践编织在一起的系统：Lean 提供 sandbox，RL 提供 search-policy 飞轮，治理提供 trace/alert/knowledge，而 newsletter 则让整个团队保持同步。